

SIMATS ENGINEERING
(SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

**CO3 ASSESSMENT TOOL 2 – CODE REVIEW & REPOSITORY
EVALUATION**

Course Code:	CSA10	Course Title:	Software Engineering
CO Assessed:	CO3	Weightage:	20%
Assessment:	Assessment Tool 2 – Code Review & Repository Evaluation	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	Surendar MG	Reg. No.:	192521374
Date:	14/08/2026	Duration:	60 minutes

Annexure A – Code Review & Repository Evaluation**Instructions to Students**

Each student has been assigned an individual problem statement. Based on the assigned problem statement, perform a Code Review and Repository Evaluation of your project. Analyze the quality of the source code, repository organization, documentation, coding practices, and overall maintainability. Provide appropriate screenshots, code snippets, diagrams, and justifications wherever applicable.

Assignment Questions**1. Problem Overview**

- Explain the assigned problem statement and summarize the implemented solution.
- Describe the project objectives and key functionalities.

2. Repository Organization

- Evaluate the repository structure, folder organization, and file naming conventions.
- Justify how the repository layout supports maintainability and collaboration.

3. Code Quality Review

- Review the source code for readability, modularity, coding standards, and documentation.
- Identify strengths and areas for improvement.

4. Version Control Evaluation

- Analyze the Git repository by reviewing commit history, branching strategy, commit messages, and repository maintenance practices.
- Explain how version control supports collaborative software development.

5. Code Testing and Validation

- Demonstrate that the application functions correctly through appropriate testing.

- Present test cases, execution results, and screenshots as evidence.

6. Repository Documentation

- Evaluate the completeness of the project documentation, including the README, installation instructions, usage guide, and dependencies.
- Suggest improvements to enhance usability and maintainability.

7. Code Review Findings and Recommendations

- Summarize the overall code review findings.
- Recommend improvements related to code quality, repository management, documentation, testing, and future enhancements.

Expected Deliverables

- Code Review Report (10–15 pages)
- Git repository link
- Screenshots of repository structure and commit history
- Code review observations with examples
- Test execution screenshots
- Recommendations for improvement

Rubrics for Calculation

Evaluation Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
Problem Analysis & Repository Organization (15%)	Clearly explains the problem, solution, and repository structure with logical organization and justification.	Good explanation with minor omissions.	Basic explanation and repository organization.	Incomplete or poorly organized repository.
Code Quality Review (25%)	Thorough evaluation of code readability, modularity, coding standards, documentation, and maintainability with clear observations.	Good code review with minor gaps.	Basic review with limited analysis.	Superficial or inaccurate code review.
Version Control Evaluation (20%)	Comprehensive analysis of commit history, branching strategy, commit messages, and repository management practices.	Good evaluation with adequate explanation.	Basic evaluation with limited evidence.	Poor or incomplete version control analysis.

Evaluation Criteria	Excellent (4)	Good (3)	Satisfactory (2)	Needs Improvement (1)
Testing & Validation (15%)	Demonstrates comprehensive testing with appropriate test cases, execution results, and supporting evidence.	Good testing with minor omissions.	Basic testing with limited evidence.	Inadequate or missing testing and validation.
Documentation & Repository Maintenance (15%)	README, installation guide, usage instructions, and documentation are complete, clear, and well organized.	Documentation is mostly complete.	Basic documentation with missing details.	Poor or insufficient documentation.
Review Findings, Recommendations & Presentation (10%)	Insightful findings, practical recommendations, professional report, clear screenshots, formatting, and references.	Good recommendations and presentation.	Basic recommendations with acceptable presentation.	Weak conclusions and poor presentation.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

ARCHITECTURE DESIGN REPORT

Smart Forest Fire Detection and Environmental
Monitoring System

2. Executive Summary

The proposed system is a continuous forest-monitoring and decision-support platform that combines IoT sensing, edge computing, AI analytics, satellite-derived observations, drone imagery, geospatial services and emergency alerting.

The architecture selected is a hybrid edge–cloud architecture with event-driven processing. Edge gateways perform local validation, buffering and basic rules so that temporary communication loss does not immediately stop monitoring. Cloud services provide scalable ingestion, AI inference, storage, dashboards, incident management and reporting.

The design directly addresses the supplied objectives: environmental monitoring, early fire detection, AI-based risk prediction, satellite/drone integration, emergency alerts, firefighting support, environmental analytics and conservation.

NASA FIRMS demonstrates the practical value of near-real-time satellite fire information, while also documenting limitations such as cloud obstruction, spatial resolution and the fact that a hotspot does not necessarily mean an entire pixel is burning. Therefore the architecture uses satellite data as one evidence source rather than an unquestionable ground truth. [1][2]

The design treats AI output as probabilistic decision support. High-severity actions remain subject to human verification and operational procedures. NIST AI RMF emphasizes trustworthy characteristics including validity/reliability, safety, security/resilience, accountability and explainability. [5]

3. Problem and Industry Context

Forest fire detection is a distributed sensing problem. Environmental conditions vary across large areas, communication links may be intermittent, and the system must distinguish real fire signals from normal environmental variation, sensor faults and unrelated heat or smoke sources.

Conventional manual observation can create a delay between ignition and awareness. The proposed platform reduces this gap by collecting environmental measurements continuously and correlating local evidence with remote-sensing observations.

Ground sensors provide high temporal resolution but limited spatial coverage. Satellite observations provide wide-area context but can be affected by cloud and spatial-resolution limitations. Drones provide targeted high-resolution evidence but require mission planning and operational constraints.

NASA FIRMS distributes near-real-time active-fire data from MODIS and VIIRS and exposes maps, alerts and data services. This supports the architectural decision to provide an external-data adapter rather than building satellite detection entirely from scratch. [1]

The system boundary includes sensing, ingestion, validation, analytics, event correlation, alerts, geospatial visualization, incident management, audit and reporting. It excludes direct autonomous control of firefighting equipment and legally binding evacuation decisions.

4. Objectives, Scope and Success Criteria

The primary objective is to provide continuous environmental awareness and earlier identification of hazardous conditions. The platform should turn raw observations into useful risk indicators, fire-event

candidates and operational alerts.

In Scope

Sensor registration; telemetry collection; edge filtering and buffering; cloud ingestion; AI risk scoring; fire-event correlation; satellite/drone integration; alerting; maps; incident tracking; environmental analytics; user access control; audit logs.

Out of Scope

Autonomous aircraft control, autonomous firefighting equipment control, legal emergency declarations, guaranteed detection under every environmental condition, and certification for life-critical use.

Success Criteria

Heterogeneous data can be ingested; timestamps and locations are preserved; network interruptions do not immediately destroy observations; risk scores are reproducible; incidents are traceable; alerts include actionable context; and partial component failures are visible to operators.

The architecture must clearly distinguish no detected fire from insufficient or unavailable data. Missing sensor data must never be silently interpreted as evidence that the forest is safe.

5. Stakeholders and Operational Scenarios

- **Forest officers:** need risk maps, incident status, historical trends and reports for conservation and response planning.
- **Emergency/firefighting teams:** need timely alerts containing location, severity, confidence and evidence, plus incident assignment and status tracking.
- **Control-room operators:** need live system health, alert queues and clear separation between environmental risk and sensor/system failure.
- **Field technicians:** need device registration, health status, configuration, diagnostics and secure update capabilities.
- **AI/data engineers:** need versioned datasets, model versions, evaluation metrics, feature definitions and feedback from verified incidents.

Operational Scenarios

Scenario 1: Temperature rises while humidity decreases and smoke concentration increases. The gateway validates readings, the stream processor creates a feature window, AI calculates risk, and the system raises an attention event.

Scenario 2: A local smoke spike overlaps spatially and temporally with a satellite hotspot. The correlation engine increases confidence and can request targeted drone confirmation.

6. Functional Requirements

ID	Requirement
FR-01	Register sensors, gateways, forest zones and device metadata.
FR-02	Collect temperature, humidity, smoke concentration and wind-speed observations.
FR-03	Validate device identity, timestamp, units, ranges, freshness and message identifiers.

ID	Requirement
FR-04	Buffer readings at the edge during connectivity interruptions.
FR-05	Calculate AI-based environmental fire-risk scores.
FR-06	Detect suspected fire events using multimodal evidence.
FR-07	Ingest satellite active-fire/thermal-anomaly observations and preserve provenance.
FR-08	Receive drone imagery/metadata and associate it with a geographic incident.
FR-09	Generate alerts containing location, severity, confidence, timestamp and evidence summary.
FR-10	Allow authorized users to acknowledge, assign, update and close incidents.
FR-11	Display live sensor status, risk maps, events and incident states.
FR-12	Generate historical environmental and fire-risk reports.
FR-13	Store model version and decision evidence for AI-generated events.
FR-14	Monitor device health, communication state and power/battery status.

7. Non-Functional Requirements and Quality Attributes

Scalability requires the system to expand from a pilot deployment to many zones and large device populations without redesigning the core workflow.

Performance requires low-latency processing for operational alerts. Exact targets should be set after field measurements because network latency and sensor sampling rates depend on deployment conditions.

Availability requires monitoring to continue through partial failures. Edge buffering, retries and redundant cloud services reduce the impact of individual outages.

Reliability requires durable message handling, stable event identifiers, idempotent consumers and audit trails so that duplicate or lost events can be detected.

Security requires device identity, encrypted transport, role-based access, least privilege, secure updates, protected secrets and audit logging. NIST SP 800-213 explicitly recommends considering cybersecurity requirements from the IoT device perspective. [3][4]

Maintainability requires modular components, versioned APIs/schemas, adapters for external systems, automated tests and observable service boundaries.

Usability requires map-first visualization, concise alerts, clear severity, freshness indicators and an acknowledgement workflow that minimizes operator ambiguity.

8. Architecture Selection

A centralized layered application would be simple for a prototype but would depend heavily on continuous connectivity and could tightly couple heterogeneous data sources.

A cloud-only microservices architecture would scale well but would assume reliable backhaul from remote forest locations. It also introduces significant operational complexity if every component

becomes a separate service.

The selected hybrid edge–cloud architecture combines local resilience with cloud scalability. Edge nodes handle device communication, validation, buffering and basic rules; cloud services handle ingestion, stream processing, AI, storage, correlation, alerts and dashboards.

Event-driven processing is selected because sensor observations, satellite detections and drone observations naturally form events. Consumers such as analytics, storage, alerting and reporting can process those events independently.

The architecture deliberately avoids excessive microservice decomposition. Services are separated where independent scaling, security, reliability or integration provides a clear benefit.

9. Overall Architecture Diagram

[Figure 2 – Proposed hybrid edge–cloud architecture diagram to be inserted here]

Figure 2. Proposed hybrid edge–cloud architecture.

The left side represents distributed data sources and edge infrastructure. The ingestion layer validates and normalizes data before publishing canonical events. The AI layer produces risk scores and detection confidence. Operational services convert those results into alerts, incidents, maps and reports.

The data layer is separated because telemetry, imagery, transactional incident records and audit information have different storage and access patterns. This improves scalability and makes future technology changes easier.

10. Component Responsibilities and Interactions

Component	Main Responsibility	Key Interaction
Sensor node	Measure environment and report health.	Gateway
Edge gateway	Authenticate, validate, buffer and forward.	Ingestion/event broker
Ingestion service	Normalize schema and publish canonical telemetry.	Stream processor/storage
Stream processor	Create temporal windows and aggregate features.	AI service
AI analytics	Predict risk and detect suspicious events.	Correlation/incident services
Correlation engine	Combine nearby multi-source evidence.	Incident/alert services
Alert service	Apply severity policy and deliver notifications.	Control room/field teams
Incident service	Manage acknowledgement, assignment and response state.	Dashboard/users
Dashboard/API	Expose maps, trends, status and incidents.	Authorized users

Canonical events should contain message ID, source/device ID, event type, timestamp, location, schema version, data-quality state and provenance. This makes replay, correlation and auditing practical.

11. Edge and IoT Architecture

[Figure 3 – Deployment zones diagram to be inserted here]

Figure 3. Deployment zones.

- Each device receives a unique identity and is registered with its location, capabilities and configuration.
- The gateway performs local range checks, timestamp checks, duplicate detection and data-quality marking.
- Durable local buffering preserves observations when cloud connectivity is unavailable.
- Basic threshold/rule checks can provide degraded-mode warnings when cloud AI is temporarily unavailable.
- Device health telemetry includes heartbeat age, battery/power state, communication quality and sensor fault indicators.
- Secure update and configuration procedures are required because field devices are physically distributed and may remain deployed for long periods.

NIST's IoT cybersecurity catalog identifies device identification, configuration, data protection, logical access, software update and cybersecurity state awareness as core capabilities. These principles directly influence the edge tier. [4]

12. AI Analytics and Data Fusion

[Figure 4 – AI-enabled detection and risk-prediction pipeline diagram to be inserted here]

Figure 4. AI-enabled detection and risk-prediction pipeline.

The AI subsystem contains two related functions: risk prediction and fire detection. Risk prediction estimates whether conditions are becoming hazardous; detection estimates whether current evidence is consistent with an emerging or active fire.

Candidate features include current and historical temperature, humidity, smoke concentration, wind speed, rates of change, rolling statistics, sensor agreement, anomaly indicators, satellite proximity and drone-derived observations.

A practical implementation can begin with interpretable tabular models such as logistic regression, decision trees or gradient-boosted models, while a separate computer-vision model handles drone imagery. A fusion layer combines model outputs and rule-based evidence.

Every prediction should store model version, timestamp, confidence, feature-set reference and data-quality state. NIST AI RMF provides a framework for managing AI risks and trustworthy characteristics such as reliability, safety, security, accountability and explainability. [5][6]

13. Satellite and Drone Integration

Satellite observations provide broad-area context and can reveal activity outside ground-sensor coverage. NASA FIRMS provides near-real-time active-fire information from MODIS and VIIRS and makes it available through maps, alerts and web services. [1]

The adapter should preserve satellite product name, acquisition time, location, detection confidence and spatial metadata. NASA notes that satellite detections have limitations involving cloud cover, view geometry and spatial resolution. [2]

Drone observations provide targeted high-resolution evidence. Each observation should be associated with mission ID, time, location/footprint and image/video metadata.

Source	Strength	Limitation
Ground IoT	High temporal resolution; local measurements	Limited spatial coverage; maintenance
Satellite	Wide-area context	Cloud/resolution/revisit limitations
Drone	Targeted visual/thermal evidence	Battery, weather and operational constraints

The architecture therefore fuses evidence instead of treating one source as ground truth. This is especially important when a single sensor is noisy or when satellite data is partially obscured.

14. Alerting and Emergency Response Workflow

[Figure 5 – Operational fire detection and response workflow diagram to be inserted here]

Figure 5. Operational fire detection and response workflow.

- Observation enters with timestamp, source and location.
- Validation checks identity, schema, range, freshness and quality.
- AI/rule services calculate risk or detection confidence.
- Spatial-temporal correlation combines independent evidence.
- Severity policy creates an alert when configured criteria are satisfied.
- Authorized operator acknowledges and assigns the incident.
- Response teams update incident state as field conditions change.
- Closed incidents become historical evidence for reporting and future model improvement.

A high-severity alert should contain location, severity, confidence, source evidence, timestamp and data freshness. The workflow is intentionally human-in-the-loop for safety-relevant decisions.

15. Data Architecture and Conceptual Data Model

[Figure 6 – Conceptual data model diagram to be inserted here]

Figure 6. Conceptual data model.

SensorReading: device, timestamp, variable, value, unit, quality and location. High-volume telemetry is suited to time-series storage.

RiskScore: zone/location reference, timestamp, model version, score, confidence and contributing feature-set reference — content to be completed as the data model is finalized.